7) Adjusted $R^2$ $= 1 - (1-R^2)\dfrac{n-1}{n-p-1}$ $n \to$ no. of observations, $p \to$ no. of predictors

8) Binary Cross Entropy (BCE) → Binary classi

$\hat{y}_i \to$ predi probab

$$BCE = \frac{-1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

$y_i \to$ actual binary l

9) Cross Entropy (CE):- → Multi-class

$y_{ij} \to$ Binary indicator (1 → sample i ∈ J, ...)

$$CE = \frac{-1}{n}\sum_{i=1}^{n}\sum_{j=1}^{k} y_{ij}\log(\hat{y}_{ij})$$

$\hat{y}_{ij} \to$ Predicted probability

10) Softmax Function → Activation func$^n$ → Converts logits to proba.

$$\text{Softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

$z = [z_1, z_2, \ldots\ldots z_k]$ : Raw scores for k classes

11) Kullback - Leibler (KL) Divergence

$P \to$ True probability func$^n$

$Q \to$ Predicted probability func$^n$

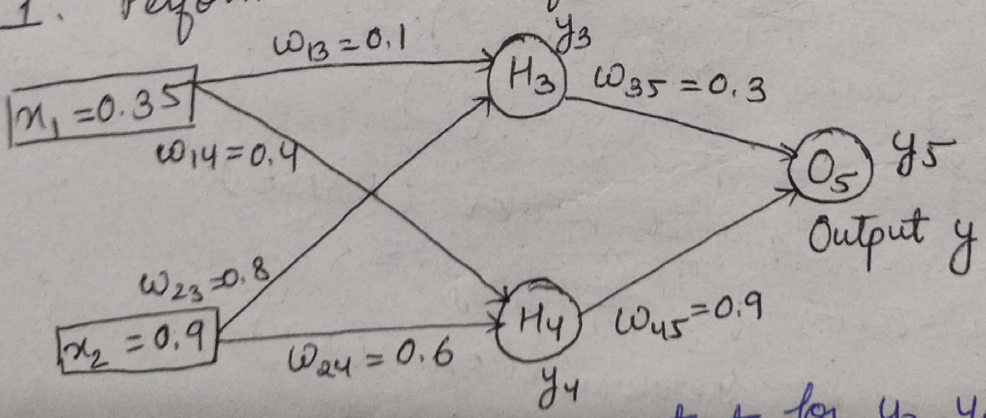
$$D_{KL}(P\|Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

$$D_{KL}(P\|Q) = H(P,Q) - H(P)$$

$D_{KL}$: Always non-ve

0 $P=Q$

Q:- Assume that the neurons have a sigmoid activation func$^n$, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.




Forward Pass: Compute output for $y_3, y_4, y_5$

$$a_j = \sum_j (w_{ij} * x_i) \qquad y_j = F(a_j) = \frac{1}{1+e^{-a_j}}$$

$x > -x \quad x>0 \qquad 2>-2 \qquad \log(am) \qquad$ If $\log_2(x)+\log_2(y) = a$

$-0.2231$ $e^x > e^{-x} \qquad 2^2 > 2^{-2} \qquad \log_2(xy) = a \qquad 2 \quad m/+n = a$

$x = 0.8 \quad e^x - e^{-x} > 0 \qquad 4 > \frac{1}{4} \qquad 2^a = xy$

$x < -x \quad x<0 \qquad -2<2$

$e^x < e^{-x} \qquad 2^{-2} < 2^2$

$e^x - e^{-x} < 0 \qquad \frac{e^{2x}-1}{e^{2x}+1}$

$e^{2x} - 1 < 0$

otherwise) $e^{2x} < 1$

$e^{2x} < e^0$

$2x < 0 \quad (x<0)$

$x = (0.8)^{\frac{1}{}}$

$$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$$
$$= (0.1 * 0.35) + (0.8 * 0.9)$$
$$= 0.755$$

$$y_3 = f(a_1) = \frac{1}{1+e^{-0.755}} = 0.68$$

$$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$$
$$= (0.4 * 0.35) + (0.6 * 0.9) = 0.68$$

$$y_4 = f(a_2) = \frac{1}{1+e^{-0.68}} = 0.6637$$

$$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$$
$$= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$$

$$y_5 = f(a_3) = \frac{1}{1+e^{-0.801}} = 0.69$$

$$\text{Error} = y_{target} - y_5 = 0.5 - 0.69$$
$$= -0.19$$

Each weight changed by:

$\Rightarrow \Delta w_{ji} = \eta \delta_j O_i$

$\Rightarrow \delta_j = O_j (1 - O_j)(t_j - O_j) \qquad j \to$ output unit

$\Rightarrow \delta_j = O_j (1 - O_j) \sum_k \delta_k w_{kj} \qquad j \to$ hidden unit

$\eta \to$ constant $\to$ learning rate

$t_j \to$ correct teacher output for unit $j$

$\delta_j \to$ error measure for unit $j$

$$\delta_5 = y(1-y)(y_{target} - y) \quad \to \text{For output unit}$$
$$= 0.69 (1-0.69) * (0.5 - 0.69)$$
$$= -0.0406$$

For hidden unit :-

$\delta_3 = y_3(1-y_3) w_{35} * \delta_5$
$= 0.68 * (1-0.68) * (0.3 * -0.0406) = -0.00265$

$\delta_4 = y_4(1-y_4) w_{45} * \delta_5$
$= 0.6637 (1-0.6637) * (0.9 * -0.0406) = -0.0082$

$\Delta w_{45} = n \delta_5 y_4 = 1 * -0.0406 * 0.6637 = -0.0269$

$w_{45}(new) = \Delta w_{45} + w_{45}(old)$
$= -0.0269 + 0.9 = 0.8731$

$\Delta w_{13} = n \delta_3 y_1 = 1 \times -0.00265 \times 0.35$
$= -9.275 \times 10^{-4}$

$w_{13}(new) = \Delta w_{13} + w_{13}(old)$
$= -9.275 \times 10^{-4} + 0.1 = 0.0991$

$\Delta w_{14} = n \delta_4 y_1 = 1 \times -0.0082 \times 0.35 = -2.87 \times 10^{-3}$

$w_{14}(new) = \Delta w_{14} + w_{14}(old)$
$= -2.87 \times 10^{-3} + 0.4 = 0.3971$

$\Delta w_{23} = n \delta_3 y_2 = 1 \times -0.00265 \times 0.9 = -2.385 \times 10^{-3}$

$w_{23}(new) = \Delta w_{23} + w_{23}(old)$
$= -2.385 \times 10^{-3} + 0.8 = 0.7976$

$\Delta w_{24} = n \delta_4 y_2 = 1 \times -0.0082 \times 0.9 = -7.38 \times 10^{-3}$

$w_{24}(new) = \Delta w_{24} + w_{24}(old)$
$= -7.38 \times 10^{-3} + 0.6 = 0.5926$

$\Delta w_{35} = n \delta_5 y_3 = 1 \times -0.0406 \times 0.68$
$= -0.0276$

$w_{35}(new) = \Delta w_{35} + w_{35}(old)$
$= -0.0276 + 0.3 = 0.2724$

$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.0991 * 0.35) + (0.7976 * 0.9) = 0.7525$

$y_3 = f(a_1) = \frac{1}{1+e^{-0.7525}} = 0.6797$

$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.3971 * 0.35) + (0.5926 * 0.9) = 0.6723$

$y_4 = f(a_2) = \frac{1}{1+e^{-0.6723}} = 0.6620$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.2724 * 0.6797) + (0.8731 * 0.6620)$
$= 0.7631$

$y_5 = f(a_3) = \frac{1}{1+e^{-0.7631}} = 0.6820$

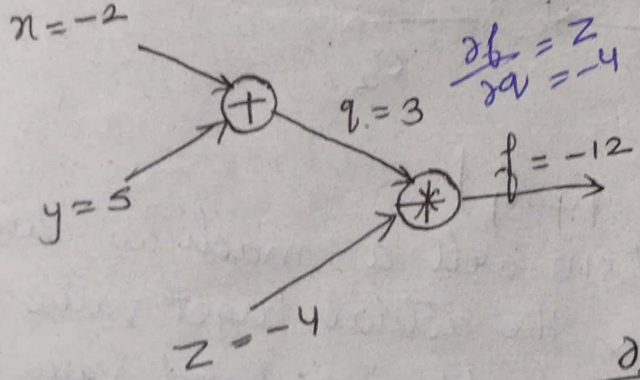
Error $= y_{target} - y_5 = 0.5 - 0.6820$
$= -0.182$

---

## Forward Pass

* Weights & bias are randomly initialized.
* The range of weights & bias is b/w 0 to 1 excluding 0 and 1

$f(x, y, z) = (x+y)z$
$q = x+y$
$f = q \times z$




* Moving forward from input layer till loss → forward pass

$\frac{\partial f}{\partial q} = z$

$\frac{\partial f}{\partial z} = q$

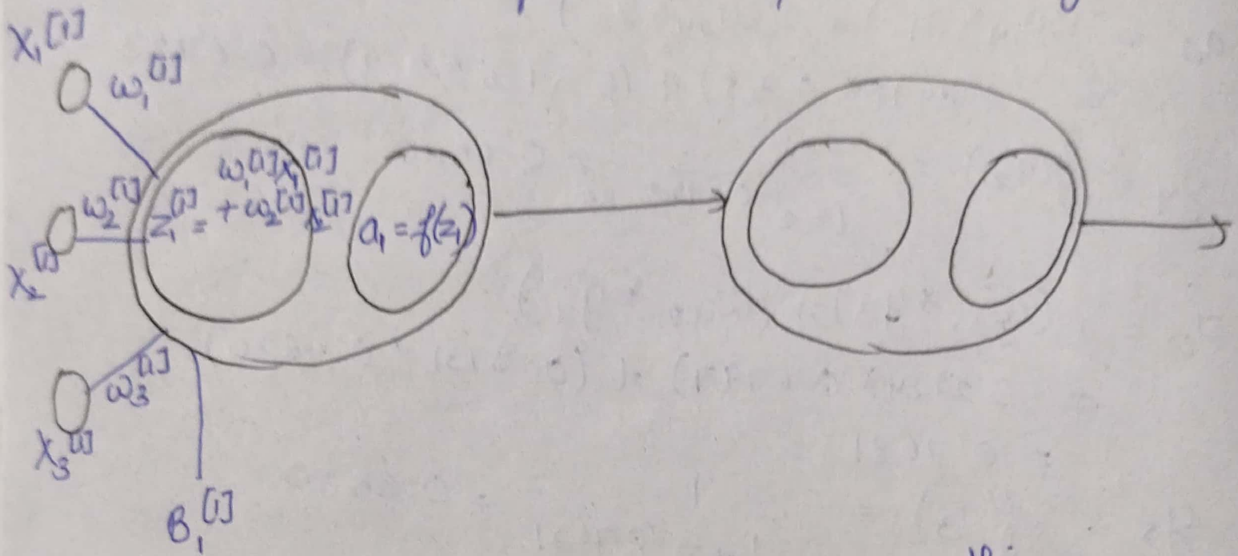
$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y}$
$= z \cdot 1 = z$

$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x}$
$= z \cdot 1 = z$

* Moving back from output to input → backward pass

$x_1^{[1]}$

* [1] → superscript represent layers
* 1 → subscript denotes neurons or units present in particular layer

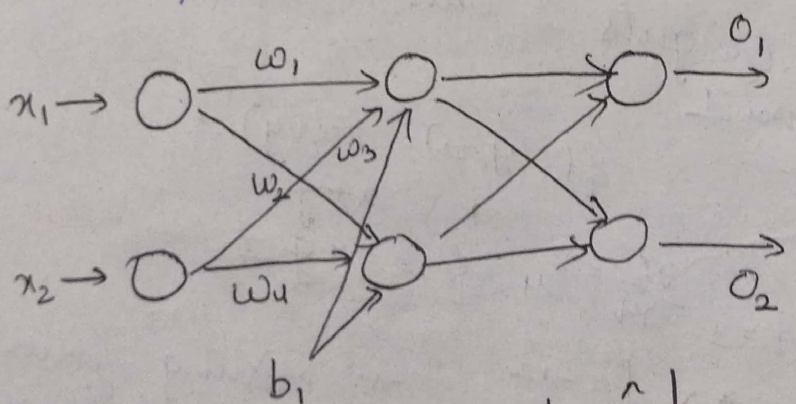



* While computing error, there are 2 things

1) Loss func^n  2) Cost Func^n

Loss function :- means for an individual sample, how the model is behaving, defined by it.

Cost function:- takes the avg | combination of all the samples & decides how a model behaves.




Mean Squared Error

$|y-\hat{y}|$

* Loss function :-
  * It quantifies how well a machine learning models prediction match the actual target value.
  * Helps us understand diff b/w predicted value & actual values.
* Error for a single training example

## Cost Function:-

* Avg of loss func$^n$ on the entire training dataset
* Types of cost func$^n$
  - → Regression
  - → Binary classification
  - → Multi-class classification

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y-\hat{y}|$$

## Gradient Descent:-

* Optimization algorithm
* Minimizes a given func$^n$ to its local minima
* It is a measure of change in weights wrt change in error (slope func$^n$)
* If slope/gradient is zero then model stops learning

$$\left[ W_{new} = W_{old} - \frac{\alpha \partial J}{\partial W_{old}} \right]$$

$\alpha$ → learning rate (0.001 or 0.0001)

$J$ → cost func$^n$

* Different kinds of Gradient Descent
  1) Stochastic GD → Every time updating the edge wt. (10 times if 10 sample (updation))
  2) Batch GD → Updates the wt only once
  3) GD

$$inh_1 = i_1w_1 + i_2w_2 = (0.05 \times 0.15) + (0.1 \times 0.4) + b_1 = 0.0475 \; \cancel{+0.35 = 0.377} \; 0.3775$$

$$inh_2 = i_1w_3 + i_2w_4 = (0.05 \times 0.25) + (0.1 \times 0.3) + b_1 = 0.0425 \; \cancel{+0.35} \; \cancel{= 0.3725} \; 0.3925$$

$$out_{h_1} = \frac{1}{1+e^{-inh_1}} = \frac{1}{1+e^{-0.3775}} = \cancel{0.5069} \; 0.5933$$

$$out_{h_2} = \frac{1}{1+e^{-inh_2}} = \frac{1}{1+e^{-0.3925}} = \cancel{0.5106}$$

$$out_{o_1} = \frac{1}{1+e^{-in}} = 0.7513$$

$$in_{o_1} = out_{h_1} \times w_5 + out_{h_2} \times w_6 + b_2$$
$$= 0.5069 \times 0.4 + 0.5106 \times 0.45$$
$$= 0.4325 \qquad 1.1059$$

$$out_{o_2} = 0.7729$$

$$in_{o_2} = out_{h_1} \times w_7 + out_{h_2} \times w_8 + b_2$$
$$= 0.5069 \times 0.5 + 0.5106 \times 0.55$$
$$= 0.5343 \qquad 1.2249$$

* In exam, if learning rate is not given then take as 0.001.

Optimization $\longrightarrow$ Gradient Descent

Loss func$^n$ $\rightarrow$ Mean squared error

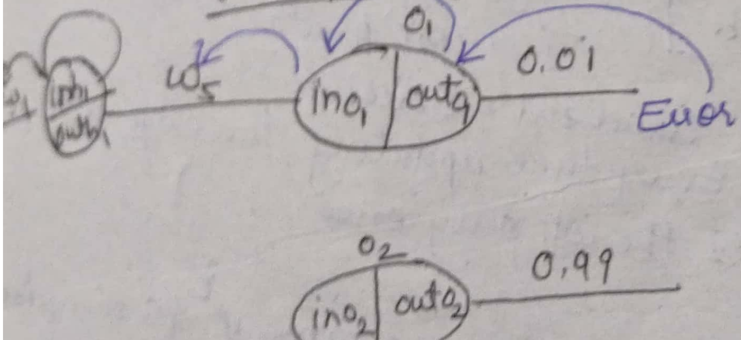
Activation func$^n$ $\rightarrow$ Sigmoid

$$Error_{o_1} = \frac{1}{2}(out_{o_1} - out_1)^2 = 0.2748 \qquad out_{o_1}^2 + out_1^2 - 2out_{o_1} out_1$$

$$Error_{o_2} = \frac{1}{2}(out_{o_2} - out_2)^2 = 0.02356$$

$$\text{Total Error} = Error_{o_1} + Error_{o_2} = 0.29837$$

Backpropagation




$$\frac{\partial Error}{\partial w_5} = \frac{\partial Error_{o_1}}{\partial w_5} + \frac{\partial Error_{o_2}}{\partial w_5}$$

$$\frac{\partial Error_{o_1}}{\partial w_5} = \frac{\partial Error_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial in_{o_1}} \times$$

$$= (out_{o_1} - out_1) \times out_{o_1}$$
$$\times \; out_{h_1}$$
$$= (0.7513 - 0.01) \times 0$$
$$\times$$
$$= 0.082167$$

$$\frac{\partial Error}{\partial w_6} = \frac{\partial Error_{o_1}}{\partial w_6} + \frac{\partial Error_{o_2}}{\partial w_6} \nearrow 0$$

$$= \frac{\partial Error_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial in_{o_1}} \times \frac{\partial in_{o_1}}{\partial w_6}$$

$$= (out_{o_1} - out_1) \times out_{o_1}(1 - out_{o_1}) \times out_{h_2}$$

$Error = \frac{1}{2n}(\quad)^2$

For simplicity purpose 2 get cancelled

$= (0.7513 - 0.01) \times 0.7513(1-0.7513) \times 0.59688$

$= 0.7413 \times 0.7513 \times 0.59688 \times 0.2487$

$= 0.08267$

$$\frac{\partial Error}{\partial w_7} = \frac{\partial Error o_1}{\partial w_7} + \frac{\partial Error O_2}{\partial w_7}$$

$$\frac{\partial Error O_2}{\partial w_7} = -0.0226$$

$$\frac{\partial Error}{\partial w_8} = \underbrace{\frac{\partial Error o_1}{\partial w_8}}_{=0} + \frac{\partial Error O_2}{\partial w_8}$$

$\frac{e^x}{1+e^x}$

$\frac{e^x(1+e^x)}{(1+e^x)^2}$

$$\frac{\partial Error O_2}{\partial w_8} = \cancel{2}(out\,o_2 - out\,2)\; out\,o_2(1-out\,o_2) \times out\,h_2$$

$$= -0.02274$$

$$\frac{\partial Error}{\partial w_1} = \frac{\partial Error O_1}{\partial w_1} + \frac{\partial Error O_2}{\partial w_1}$$

$\frac{\partial in_{o_1}}{\partial w_5}$

$$\frac{\partial Error O_1}{\partial w_1} = \frac{\partial Error O_1}{\partial out\,o_1} \times \frac{\partial out\,o_1}{\partial in\,o_1} \times \frac{\partial in\,o_1}{\partial out\,h_1} \times \frac{\partial out\,h_1}{\partial in\,h_1} \times \frac{\partial in\,h_1}{\partial w_1}$$

$(1-out\,o_1)$.

$= \cancel{2}(out\,o_1 - out_1)\; out\,o_1(1-out\,o_1)\; w_5 \times out\,h_1(1-out\,h_1) \times i_1$

$= (0.7513 - 0.01) \times 0.7513(1-0.7513) \times 0.4 \times 0.5933(1-0.5933) \times 0.05$

$513(1-0.7513)$

* If it is fully connected layer then a neuron at $(l-1)$th layer will connect to all the neurons in the next layer

$$\frac{\partial Error O_2}{\partial w_1} = \frac{\partial Error O_2}{\partial out\,o_2} \times \frac{\partial out\,o_2}{\partial in\,o_2} \times \frac{\partial in\,o_2}{\partial out\,h_1} \times \frac{\partial out\,h_1}{\partial in\,h_1} \times \frac{\partial in\,h_1}{\partial w_1}$$

$= (out\,o_2 - out\,2) \times out\,o_2(1-out\,o_2) \times w_7 \times out\,h_1(1-out\,h_1) \times i_1$

$$\frac{\partial Error}{\partial w_2} = \frac{\partial Error_{o_1}}{\partial w_2} + \frac{\partial Error_{o_1}}{\partial w_2}$$

$$\frac{\partial Error_{o_1}}{\partial w_2} = \frac{\partial Error}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial in_{o_1}} \times \frac{\partial in_{o_1}}{\partial out_{h_1}} \times \frac{\partial out_{h_1}}{\partial in_{h_1}} \times \frac{\partial in_{h_1}}{\partial w_2}$$

Variations:—

* Different loss/cost function
* Activation func$^n$ may vary

$$\frac{\partial Error}{\partial w_3} = \frac{\partial Error_{o_1}}{\partial w_3} + \frac{\partial Error_{O_2}}{\partial w_3}$$

$$\frac{\partial Error_{o_1}}{\partial w_3} = \frac{\partial Error_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial in_{o_1}} \times \frac{\partial in_{o_1}}{\partial out_{h_2}} \times \frac{\partial out_{h_2}}{\partial in_{h_2}} \times \frac{\partial in_{h_2}}{\partial w_3}$$

$$\frac{\partial Error_{o_2}}{\partial w_3} = \frac{\partial Error_{o_2}}{\partial out_{o_2}} \times \frac{\partial out_{o_2}}{\partial in_{o_2}} \times \frac{\partial in_{o_2}}{\partial out_{h_2}} \times \frac{\partial out_{h_2}}{\partial in_{h_2}} \times \frac{\partial in_{h_2}}{\partial w_3}$$

$$w_1 = w_1 - \alpha \frac{\partial Error}{\partial w_i} \qquad \alpha \rightarrow \text{Learning Rate}$$

Derivative of sigmoid → 0 to 0.25
Outputs are not zero centred

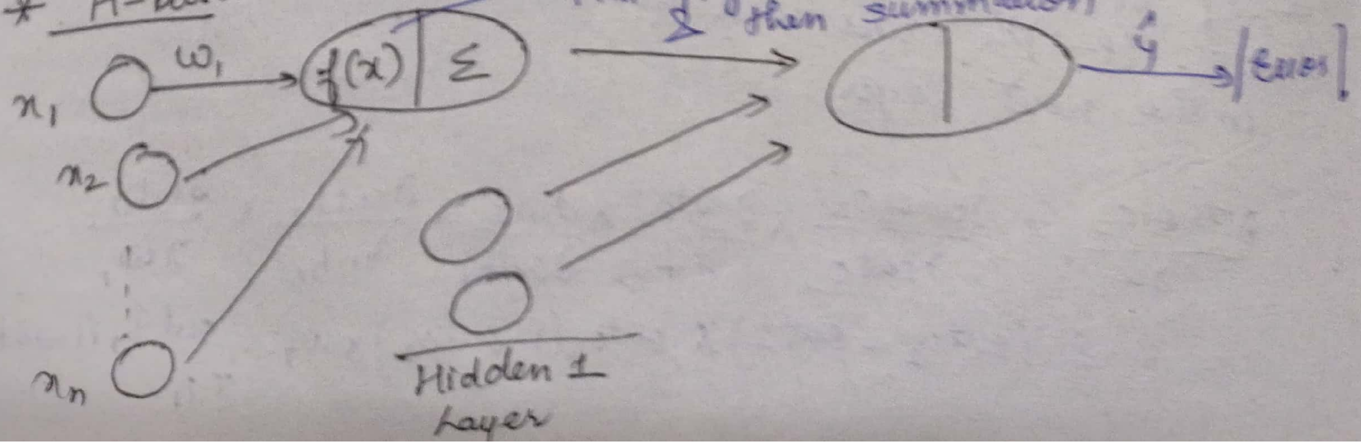
ReLU → non linear (since different properties for $x \geq 0$ and $x \leq 0$)

Leaky Relu → $x$, $\alpha x$

* H-based Activation

Reduced overfitting

First apply activation func$^n$ & then summation



| $y$ | $\hat{y}$ |
| --- | --- |
| 1000 | 950 |
| 900 | 800 |
| 800 | 760 |

$SAE = 50 + 100 + 40$
$= 190$

$MAE = \frac{190}{3} = 63.33$

$SSE = (50)^2 + (100)^2 + (40)^2$
$= 2500 + 10000 + 1600$
$= 14100$

$RMSE = \sqrt{\frac{14100}{3}} = \sqrt{4700} = 68.56$

* Cost funcⁿ may be of different types. It's derivative, funcⁿ & usage is imp.
* Optimizers →
* Meta learning → Training a model using small data
(N-way k shot learning)
* Augmentation Techniques → Variation in Data
* Usually augmentation techniques are used to bring diversity in the data/increase the sample size by using orientation, flipping, color space change
* Recently augmentation techniques are used to regularize the model.

## Scope of Improvement

- More data may be required
- Data needs to have more diversity
- Algorithm needs longer training

Epoch → atleast once the training sample is send for backpropagation.

## Dataset Splitting

* Training set → which you run your learning algorithm on.
* Dev or Validation Set → Which you use to tune parameters, select features & make other decisions regarding the learning algorithm. Sometimes also called the hold-out cross validation set.

Test Set → which you use to evaluate the performance of the algorithm, but not to make any decisions regarding what learning algo or parameters to use

* If you are working with images, try to bring gaussian distribution into consideration.

* Batch GD :- H

$\theta = \theta - \eta \nabla J(\theta)$  $\theta$ → are the parameters which need to be updated

* Guarantees convergence to the global minimum
* Stable & smooth updates
* slow for large datasets
* Requires loading the entire dataset into memory

Loss → for individual samples.
Cost →

* Stochastic GD :- For every sample, the weights will be updated

$\theta = \theta - \eta \cdot \nabla J(\theta, x^i y^i)$

* Faster iteration speed
* Can escape local minima (due to noisy updates)
* Suitable for online learning
* Convergence can be noisy and less stable
* Requires careful tuning of the learning rate

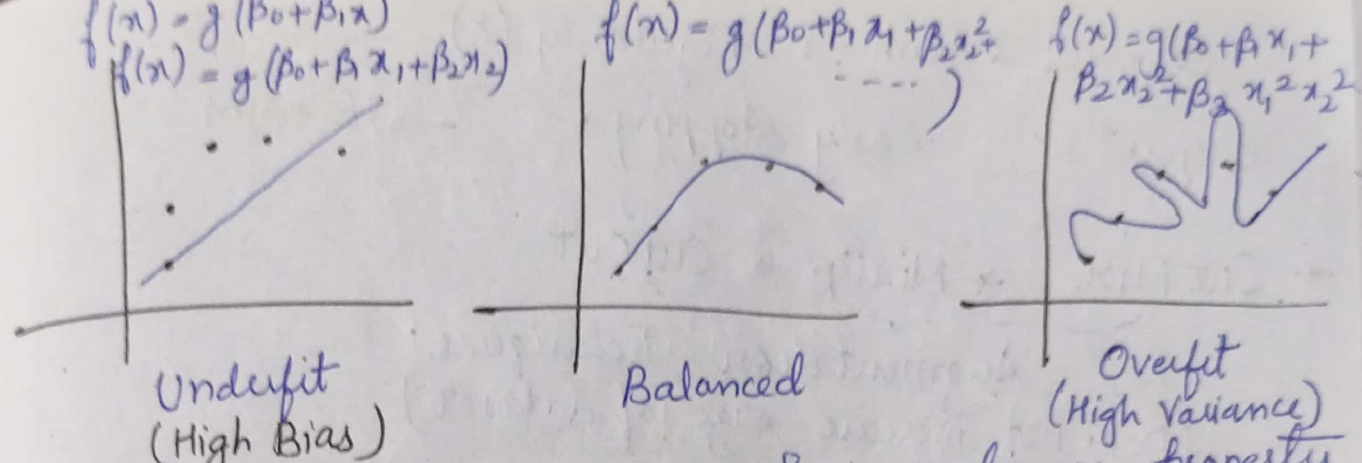
Mini-Batch GD :-

$\theta = \theta - \eta . \frac{1}{m}$

Overfitting & Underfitting  Balanced

↳ Model becomes so complicated that during testing time it fails

(During training → Min Error
Testing → Max.




Linear Regression → Data is having linear property (Assumption)

* Function does not get fitted or the funcⁿ does not match the pattern present in the data
* This may be due to high assumption which leads to bias.

High Bias → Underfit (Training error is high). Model is too simple to capture the patterns in the data.

High Variance – Overfit (Model fails on test/unseen data)

Bias = $E[\hat{y}] - y$

Variance = $E\left((\hat{y} - E(y))^2\right)$

* If your validation error is very high as compared to training error
* It might happen that your training & validation error are min. but test → max. ⇒ Overfitting
* If model is underfit, then training, validation & test error are very high

Regularization :- (Trying to restrict learning of model) Poisonous data → Model learns on wrong data

* L1 Regularization (Lasso)

Adds the absolute value of the coefficients as a penalty term to the loss funcⁿ

$\hat{y}$

Sigmoid & tanh → Regularity (x)

Dropout ✓ Early stopping ✓ → Exams

* Cut Mix * MixUp * CutOut

Augmentation Techniques.
(Does not increase size of dataset)

## Bayesian Framework

$$P(A/B) = \frac{P(A \cap B)}{P(B)}$$ Conditional Probability

$$P(B/A) = \frac{P(B \cap A)}{P(A)}$$

$P(A \cap B) = P(B \cap A)$

$\Rightarrow P(A/B)\,P(B) = P(B/A)\,P(A)$

$$\Rightarrow \boxed{P(B/A) = \frac{P(A/B)\,P(B)}{P(A)}}$$ ← Naive Bayes Equation

$A = \{\theta_1, \theta_2, \theta_3 \ldots \theta_n\}$; set of Independent Events

$C_i$ = Labels/Targets

$$P(C_i|\theta_1, \theta_2, \ldots \theta_n) = \frac{P(\theta_1, \theta_2, \ldots \theta_n | C_i)\,P(C_i)}{P(\theta_1, \theta_2, \ldots \theta_n)}$$

$$= \frac{P(\theta_1/C_i)\,P(\theta_2/C_i) \ldots\ldots P(\theta_n|C_i)\,P(C_i)}{P(\theta_1)\,P(\theta_2) \ldots\ldots P(\theta_n)}$$

$$P(C_i|\theta_1, \theta_2, \ldots \theta_n) = P(\theta_1/C_i)\,P(\theta_2/C_i) \ldots\ldots P(\theta_n|C_i)\,P(C_i)$$

Outlook

| Temp | Yes | No | P(Yes) | P(No) |
|---|---|---|---|---|
| Hot | 2 | 2 | 2/9 | 2/5 |
| Mild | 4 | 2 | 4/9 | 2/5 |
| Cool | 3 | 1 | 3/9 | 1/5 |
| Total | 9 | 5 | 100% | 100% |

Humidity

| | Yes | No | P(yes) | P(No) |
|---|---|---|---|---|
| High | 3 | 4 | 3/9 | 4/5 |
| Normal | 6 | 1 | 6/9 | 1/5 |
| Total | 9 | 5 | 100% | 100% |

Windy

| | Yes | No | P(yes) | P(No) |
|---|---|---|---|---|
| Weak | 6 | 2 | 6/9 | 2/5 |
| Strong | 3 | 3 | 3/9 | 3/5 |
| Total | 9 | 5 | 100% | 100% |

P.T.W Probability that match will be played "today" if it is Sunny, Cool, Normal & Weak?

$P(yes) = \frac{\text{No. of yes}}{\text{Total no. of matches}} = 9/14 \qquad P(No) = 5/14$

$P\left(\frac{\text{sunny}}{\text{yes}}\right) = \frac{2}{9} \qquad P(\text{cool}/\text{yes}) = \frac{3}{9} \qquad P(\text{Normal}/\text{yes}) = 6/9$

$P(\text{Weak}/\text{yes}) = 6/9$

$P(\text{sunny}/No) = \frac{3}{5} \qquad P(\text{Cool}/No) = 1/5 \qquad P(\text{Normal}/No) = 1/5 \qquad P\left(\frac{\text{Weak}}{No}\right) = \frac{2}{5}$

$P(\text{yes}/\text{today}) = P\left(\frac{\text{sunny}}{\text{yes}}\right) P\left(\frac{\text{Cool}}{\text{yes}}\right) P\left(\frac{\text{Normal}}{\text{yes}}\right) P(\text{weak}/\text{yes}) P(\text{yes})$

$= \frac{2}{9} \times \frac{3}{9} \times \frac{6}{9} \times \frac{6}{9} \times \frac{9}{14} = \cancel{0.0317}\ 0.021164$

$P(No/\text{today}) = \cancel{0.00228}\ 0.003429$

Note:— In exam, if probability is asked then normalize the probability

Else if only category is asked then above probabilities are ok.

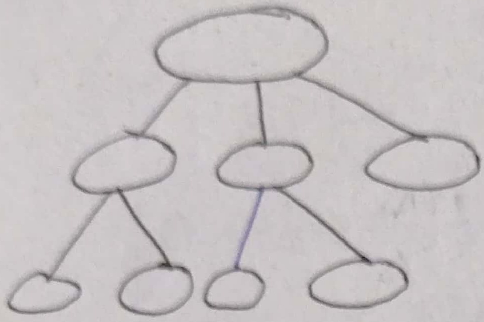
$P(\text{Yes}/\text{today}) + P(No/\text{today}) = 1$

0

$P(\text{Yes}/\text{today}) =$

# Decision Tree

Basic algorithm (a greedy algorithm)



* Leaf Node represent output or class
* Internal / Root Node represent Test condⁿ

3 Famous algo:-
1) Information Gain
2) Gain Ratio
3) Gini Index

$$H(S) = \sum p(x) \left(\log_2 \cancel{p(x)} \cancel{\left(\frac{1}{p(x)}\right)} \cancel{p(x)} \frac{1}{P(x)}\right)$$

* If entropy is maximum then the impurity is at the peak
* If entropy = 1 then classes will have equal probability.
* If entropy = ~~0.25~~ 1 for binary class then both classes will have equal probability.

$$Entropy(S) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log\left(\frac{5}{14}\right) = 0.940$$

$$P(S_{weak}) = 8/14 \qquad P(S_{strong}) = 6/14$$

$$Entropy(S_{weak}) = -6/8 \log\left(\frac{6}{8}\right) - \left(\frac{2}{8}\right)\log\left(\frac{2}{8}\right) = 0.811$$

$$Entropy(S_{strong}) = -\frac{3}{6}\log_2\left(\frac{3}{6}\right) - \left(\frac{3}{6}\right)\log_2\left(\frac{3}{6}\right) = 1$$

$$IG(S, wind) = Entropy(S) - \sum_{i=0}^{n} p(x) * Entropy(x)$$

$$= 0.940 - \left(\frac{8}{14} \times 0.811 + \frac{6}{14}\right)$$

$$= 0.048$$

Outlook

| | Yes | No | P(yes) | P(No) |
|---|---|---|---|---|
| Rainy | 2 | 3 | 2/9 | 3/5 |
| Overcast | 4 | 0 | 4/9 | 0 |
| Sunny | 3 | 2 | 3/9 | 2/5 |
| | 9 | 5 | | |

~~Entropy (Overcast)~~ Entropy(S) = 0.940

$P(S_{Rainy}) = 5/14$ $\quad P(S_{overcast}) = 4/14$

$P(S_{sunny}) = 5/14$

$Entropy(S_{Rainy}) = -\frac{2}{5}\log\left(\frac{2}{5}\right) - \left(\frac{3}{5}\right)\log(3/5)$

$= \cancel{0.2923}\ 0.971$

$Entropy(S_{overcast}) = -\frac{4}{4}\log(4/4) - (0/4)\log_2(0/4) = 0$

$Entropy(S_{sunny}) = -\frac{3}{5}\log_2(3/5) - \frac{2}{5}\log_2(2/5) = 0.971$

If entropy = 0 then pure class

$IG(S, Outlook) = Entropy(S) - \sum_{i=0}^{n} p(x) * Entropy(x)$

$= 0.940 - \left(\frac{5}{14}\times 0.971 + 0 + \frac{5}{14}\times 0.971\right)$

$= 0.2464$

$P(S_{Hot}) = 4/14 \quad P(S_{Mild}) = \frac{6}{14} \quad P(S_{cool}) = \frac{4}{14}$

$Entropy(S_{Hot}) = -\frac{2}{4}\log\left(\frac{2}{4}\right) - \frac{2}{4}\log\left(\frac{2}{4}\right) = 1$

$Entropy(S_{Mild}) = -\frac{4}{6}\log\left(\frac{4}{6}\right) - \frac{2}{6}\log\left(\frac{2}{6}\right) = 0.9183$

$Entropy(S_{cool}) = -\frac{3}{4}\log\left(\frac{3}{4}\right) - \frac{1}{4}\log\left(\frac{1}{4}\right) = 0.8113$

$IG(S, Temp) = Entropy(S) - \sum_{i=0}^{n} p(x) * Entropy(x)$

$= 0.940 - \left(\frac{4}{14}\times 1 + \frac{6}{14}\times 0.9183 + \frac{4}{14}\times 0.8113\right)$

$= 0.0289$

$P(\text{High}) = \frac{7}{14}$ $\qquad$ $P(\text{Normal}) = \frac{7}{14}$

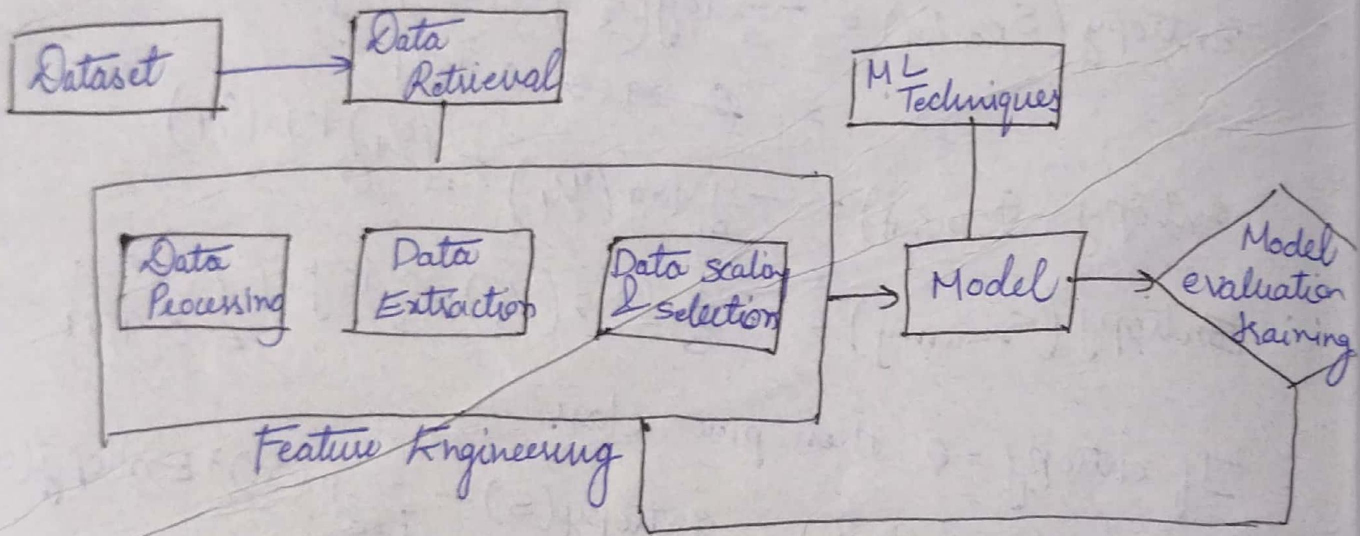
$$\text{Entropy}(S_{\text{High}}) = -\frac{3}{7}\log\left(\frac{3}{7}\right) - \frac{4}{7}\log\left(\frac{4}{7}\right) = 0.9852$$

$$\text{Entropy}(S_{\text{Normal}}) = -\frac{6}{7}\log\left(\frac{6}{7}\right) - \frac{1}{7}\log\left(\frac{1}{7}\right) = 0.5917$$

$$IG(S, \text{Humidity}) = \text{Entropy}(S) - \sum p(x)\,\text{Entropy}(x)$$

$$= 0.940 - \left(\frac{7}{14} \times 0.9852 + \frac{7}{14} \times 0.5917\right)$$

$$= 0.1516$$




Test data → unseen

Module & Library diff.

Module ↓ Contains a single file

Library → Collection of modules

head() → 1st 5 rows

info() → data type